AI ENGINEERING MASTERY BUCKET · MODULE 18

LLMOps & Production AI

Running AI systems reliably at real scale — LLMOps, deployment, model serving, API gateways, rate limits, retries, caching, streaming, batch/async inference, model fallbacks, reliability, and scalability.

DIFFICULTY LEVEL
Intermediate–Advanced

ESTIMATED READING TIME
55–70 minutes

ESTIMATED PRACTICE TIME
90–120 minutes

PREREQUISITES
Modules 01–17

LEARNING OUTCOMES

By the end of this module, you will be able to: describe the LLMOps lifecycle for deploying and running AI systems; handle rate limits with retries and backoff correctly; apply caching and streaming to cut cost and latency; choose between real-time, batch, and async inference; design fallback strategies for model failures; and reason about reliability and scalability for production AI systems.

Table of Contents

1. Introduction

What is this topic?

Modules 16 and 17 covered knowing your system works and watching it in production. This module covers the operational engineering that keeps it running reliably, efficiently, and affordably at real scale — the infrastructure layer beneath everything else in this course.

Why is it important?

A prompt that works perfectly in a single test call can still fail in production if it hits a rate limit with no retry logic, times out with no fallback, or costs 10x more than necessary because nothing is cached. LLMOps is where correctness meets reliability at scale.

Why should AI Engineers learn it?

These are the practices that separate a working demo from a production system real users depend on — handling failures gracefully, controlling cost at volume, and keeping latency acceptable under real load.

Where is it used in the real world?

Every AI product serving real traffic needs some combination of retries, caching, fallbacks, and thoughtful inference-mode choices — this is standard infrastructure, not an advanced edge case.

Why is it relevant today?

As more AI features move from prototype to production-critical, LLMOps has emerged as its own discipline — borrowing heavily from traditional DevOps/SRE practices, adapted for the specific failure modes and cost structure of LLM APIs.

> **Note**
> Many concepts here (retries, caching, gateways) exist in traditional software engineering too — this module focuses on what's specifically different or especially important for LLM-based systems.

2. Before You Begin

RECAP

Cost and latency optimization (Module 09, 3.4-3.6) — this module operationalizes those same concerns into concrete production infrastructure patterns.

RECAP

Production monitoring (Module 17, 3.8) — the metrics from Module 17 are exactly what tells you if this module's practices (caching, fallbacks) are actually working.

Tip

Implement retries with backoff and basic caching early — they're cheap to add and provide an outsized reliability/cost benefit relative to the effort required.

3. Main Lesson

Each concept follows: **Definition → Simple Explanation → Analogy → Everyday Example → Why It Matters → Common Mistakes → Best Practices → Summary.**

⚙3.1 LLMOps

DEFINITION

LLMOps is the discipline of operationalizing LLM-based applications — the practices, tooling, and processes for deploying, monitoring, and maintaining them reliably in production.

SIMPLE EXPLANATION

LLMOps is to LLM applications what DevOps/MLOps is to traditional software/ML systems — it covers everything from Modules 16-17 (evaluation, observability) plus this module's deployment, reliability, and scaling concerns, as one connected practice.

> 💡 *Like the difference between writing a recipe (the prompt/model logic) and running a restaurant kitchen that reliably serves that recipe to hundreds of customers a night, on time, every time.*

EVERYDAY EXAMPLE

A team's LLMOps practice might include: automated evaluation on every change (Module 16), production tracing (Module 17), and this module's rate-limit handling and fallback strategies, all working together.

WHY IT MATTERS

Without deliberate LLMOps practices, teams tend to discover reliability and cost problems only after they've already impacted real users.

COMMON MISTAKES

Treating LLMOps as an afterthought to bolt on later, rather than a practice built in from a project's early stages.

BEST PRACTICES

Build core LLMOps practices (evaluation, tracing, basic reliability patterns) into a project from the start, even in simple form.

SUMMARY

LLMOps is the umbrella discipline for running LLM applications reliably in production — this module covers its core operational patterns.

🚀3.2 Deployment & Model Serving

DEFINITION

Deployment is the process of making an AI application available to real users; model serving specifically refers to how the underlying model is made available to respond to requests (via a managed API, Module 12, or self-hosted infrastructure, Module 11, 3.4).

SIMPLE EXPLANATION

For most teams using closed APIs (Module 12), "model serving" is handled entirely by the provider — deployment is really about deploying your application code that calls that API. Teams using open-weight models (Module 11) need to additionally manage the model-serving layer itself.

> 💡 *Like the difference between opening a restaurant that buys ingredients from a supplier (using a managed API) versus one that also runs its own farm (self-hosting the model) — both can serve great food, but one has significantly more infrastructure to manage.*

EVERYDAY EXAMPLE

Deploying a chatbot application to a cloud server that calls the Anthropic API (simple deployment, managed serving) versus deploying both the application and a self-hosted Llama model on your own GPU infrastructure (deployment plus full serving responsibility).

WHY IT MATTERS

Knowing which layer you're actually responsible for shapes your entire operational burden — managed API users skip most of Section 3.9's scalability concerns for the model itself.

COMMON MISTAKES

Underestimating the operational complexity of self-hosted model serving when open-weight models (Module 11) seemed attractive purely for cost reasons.

BEST PRACTICES

Default to managed APIs unless you have a specific, justified reason (data residency, extreme cost sensitivity, Module 11) to take on model-serving responsibility yourself.

SUMMARY

Deployment and model serving define who's responsible for what — managed APIs shift most serving complexity to the provider, self-hosting takes it on yourself.

3.3 API Gateways

DEFINITION

An API gateway is a layer that sits between your application and the underlying model API(s), centralizing concerns like authentication, rate limiting, routing, and logging.

SIMPLE EXPLANATION

Instead of every part of your application calling provider APIs directly, requests go through one gateway that can enforce consistent policies, route to different providers/models (Module 11, 3.9's model routing), and centralize the tracing from Module 17.

> 💡 *Like a hotel's front desk that routes every guest request to the right department — instead of guests wandering the building trying to find housekeeping or room service themselves, one consistent entry point handles routing and policy.*

EVERYDAY EXAMPLE

An internal API gateway that all of a company's AI features call through, which enforces per-team rate limits, logs every request centrally, and can swap the underlying model provider without every application needing to change.

WHY IT MATTERS

A gateway centralizes cross-cutting concerns (Section 3.4's rate limits, Module 17's tracing) so every individual application doesn't need to reimplement them separately.

COMMON MISTAKES

Adding gateway complexity for a small, single-application project where it provides little benefit over calling the API directly.

BEST PRACTICES

Introduce an API gateway once you have multiple applications/teams sharing model access, or need centralized policy enforcement — not necessarily for a single small project.

SUMMARY

API gateways centralize cross-cutting operational concerns across multiple applications — valuable at organizational scale, often unnecessary for a single small project.

3.4 Rate Limits, Retries & Exponential Backoff

DEFINITION

Rate limits are provider-imposed caps on requests per time period; retries are re-attempting a failed request; exponential backoff is progressively increasing the wait time between retry attempts to avoid overwhelming an already-struggling system.

SIMPLE EXPLANATION

When a request hits a rate limit or transient error, immediately retrying at full speed often makes things worse. Exponential backoff waits progressively longer between attempts (1s, 2s, 4s, 8s...), giving the system room to recover.

```
Attempt 1: fails → wait 1s
Attempt 2: fails → wait 2s
Attempt 3: fails → wait 4s
Attempt 4: succeeds ✓
```

💡 *Like waiting progressively longer between attempts to call a busy customer service line — calling back immediately and repeatedly just adds to the congestion; waiting a bit longer each time gives the queue a chance to clear.*

EVERYDAY EXAMPLE

A batch job (Section 3.7) hitting a rate limit mid-run automatically waits with exponential backoff and retries, instead of crashing the entire job on the first rate-limit error.

WHY IT MATTERS

Without retry/backoff logic, a single transient error or rate limit can crash an entire request or batch job that would have succeeded with a brief, sensible wait.

COMMON MISTAKES

Retrying immediately and repeatedly at full speed with no backoff, which can worsen rate-limit situations rather than resolve them.

BEST PRACTICES

Implement exponential backoff with a maximum retry count and a maximum total wait time; only retry genuinely transient errors, not permanent ones (like invalid input).

SUMMARY

Retries with exponential backoff turn transient failures into recoverable hiccups instead of hard crashes — one of the highest-value, lowest-effort reliability patterns in this module.

💾 3.5 Caching

DEFINITION

Caching is storing and reusing previous results for repeated or similar requests, avoiding redundant (and costly) model calls.

SIMPLE EXPLANATION

If the same or a very similar prompt is likely to recur (common questions, repeated document analysis), caching the response — or even just the expensive parts, like retrieved context, Module 07 — can dramatically cut cost and latency for repeat requests.

💡 *Like a restaurant kitchen prepping common ingredients ahead of time instead of starting from scratch for every single order — the popular dishes get served much faster because the repetitive work was already done once.*

EVERYDAY EXAMPLE

Caching the answer to a frequently-asked support question so the 500th user asking it gets an instant cached response instead of triggering another full model call.

WHY IT MATTERS

Caching can be one of the single biggest cost and latency wins available, especially for applications with genuinely repetitive query patterns.

COMMON MISTAKES

Caching responses to genuinely dynamic or personalized queries where reuse would actually be incorrect, or caching indefinitely without any invalidation strategy for stale data.

BEST PRACTICES

Cache when queries genuinely repeat; set sensible cache expiration; consider semantic caching (matching similar, not just identical, queries) for even broader coverage.

SUMMARY

Caching avoids redundant model calls for repeated requests — a high-leverage cost and latency optimization when query patterns actually repeat.

🌊3.6 Streaming

DEFINITION

Streaming is returning a model's response incrementally, token by token, as it's generated, rather than waiting for the entire response to complete before sending anything back.

SIMPLE EXPLANATION

This directly builds on Module 01's generation loop (3.3) — since the model already generates one token at a time, streaming just means passing each token to the user immediately instead of buffering the whole response first.

> 💡 *Like watching subtitles appear as someone speaks, versus waiting for them to finish their entire speech before seeing any text at all — same total content, dramatically different experience of waiting.*

EVERYDAY EXAMPLE

A chat interface where text visibly appears word by word as Claude generates it, rather than a blank screen followed by the entire response appearing at once.

WHY IT MATTERS

Streaming doesn't reduce total generation time, but it dramatically improves *perceived* latency (Module 09, 3.6) — users start reading immediately instead of staring at a loading spinner.

COMMON MISTAKES

Not using streaming for user-facing, interactive features where perceived responsiveness matters most.

BEST PRACTICES

Use streaming for any interactive, user-facing text generation; it's less relevant for structured output that needs full validation (Module 04) before use.

SUMMARY

Streaming shows generation as it happens — a near-free improvement to perceived latency for interactive, user-facing applications.

📦⏳3.7 Batch & Async Inference

DEFINITION

Batch inference processes many requests together as a group, often at lower cost and without real-time latency requirements; async inference submits a request and retrieves the result later, rather than waiting synchronously.

SIMPLE EXPLANATION

Not every request needs an instant response. For large-volume, non-interactive workloads

(processing 10,000 documents overnight), batch or async modes trade immediacy for cost savings and higher throughput — many providers offer discounted pricing for batch processing specifically.

> 💡 *Like the difference between same-day rush shipping (real-time inference) and standard shipping (batch) — standard shipping costs less and is perfectly fine when you don't need it there today.*

EVERYDAY EXAMPLE

Running sentiment classification (Module 03, 3.3) on a week's worth of accumulated customer reviews as an overnight batch job, rather than processing each one in real time as it arrives.

WHY IT MATTERS

Using real-time, synchronous inference for genuinely non-time-sensitive, high-volume work wastes money and adds unnecessary application complexity (handling live rate limits, Section 3.4, at high volume).

COMMON MISTAKES

Defaulting to real-time synchronous calls for large, non-interactive workloads that would be cheaper and simpler as a batch job.

BEST PRACTICES

Match the inference mode to the actual latency requirement: real-time only for interactive user-facing needs; batch/async for large, non-time-sensitive workloads.

SUMMARY

Batch and async inference trade immediacy for cost and throughput — the right default for large, non-interactive workloads.

🔀3.8 Model Fallbacks

DEFINITION

A model fallback is a pre-configured secondary model or provider that a system automatically switches to if the primary one fails or is unavailable.

SIMPLE EXPLANATION

Even reliable providers occasionally have outages or degraded performance — a fallback (another provider from Module 12, or a different model from Module 11) keeps your application working, possibly with reduced capability, instead of failing completely.

> 💡 *Like a backup generator that kicks in automatically during a power outage — not necessarily as good as the main power grid, but keeps critical systems running instead of going completely dark.*

EVERYDAY EXAMPLE

A support chatbot that falls back from Claude to a secondary provider (via an aggregator, Module 12, 3.4) if the primary provider returns errors for more than a few consecutive requests.

WHY IT MATTERS

Without a fallback, a single provider outage becomes a complete application outage — fallbacks convert a hard failure into graceful, reduced-capability degradation (Module 34).

COMMON MISTAKES

Never testing the fallback path, discovering it's broken only during an actual real outage when it's needed most.

BEST PRACTICES

Regularly test fallback paths even when the primary is healthy; log every fallback activation (Module 17) to track primary-provider reliability over time.

SUMMARY

Model fallbacks keep an application running through a primary provider's failure — essential for genuinely production-critical AI features.

🏗️3.9 Reliability & Scalability

DEFINITION

Reliability is a system's ability to consistently work correctly despite failures; scalability is its ability to handle growing load (more users, more requests) without breaking down or degrading unacceptably.

SIMPLE EXPLANATION

This module's other concepts (retries, caching, fallbacks) are the concrete techniques; reliability and scalability are the resulting properties of a system built with those techniques applied thoughtfully together.

> 💡 *Like a bridge's load rating and safety margin — not any single design choice, but the emergent property of every structural decision (materials, redundancy, maintenance) working together under real-world stress.*

EVERYDAY EXAMPLE

A system that handles 10x its normal traffic during a launch event without falling over — the result of caching (Section 3.5) reducing redundant load, fallbacks (Section 3.8) handling provider hiccups, and batch/async (Section 3.7) absorbing non-urgent work.

WHY IT MATTERS

Reliability and scalability are what determine whether an AI feature can graduate from a small pilot to a genuinely large-scale, business-critical product.

COMMON MISTAKES

Only testing a system at small, controlled scale and assuming it will scale linearly, without validating behavior under real load and failure conditions.

BEST PRACTICES

Load-test before scaling up traffic; apply this module's techniques (retries, caching, fallbacks) together rather than in isolation; use Module 17's monitoring to validate reliability continuously, not just once.

SUMMARY

Reliability and scalability are the emergent result of this module's techniques applied together — the ultimate measure of whether a system is truly production-ready.

4. Visual Learning

🔁 Retry with Exponential Backoff

```
Request → Fail
   |
   ▼ wait 1s
Retry 1 → Fail
   |
   ▼ wait 2s
Retry 2 → Fail
   |
   ▼ wait 4s
Retry 3 → Success ✅  (or: max retries reached → fallback, Section 3.8)
```

🌳 Inference Mode Decision Tree

```
Choosing an Inference Mode
├── Need it right now, interactive? → Real-time + Streaming
├── Large volume, not urgent? → Batch Inference
└── Submit now, check later? → Async Inference
```

📊 Production Pattern Quick Reference

Pattern	Solves
Retries + backoff	Transient errors, rate limits
Caching	Redundant cost/latency on repeated queries
Streaming	Perceived latency for interactive use
Batch/async inference	Cost and throughput at high volume
Model fallbacks	Provider outages or degraded service

5. Real World Applications

Product	How these fundamentals show up
Consumer chatbots	Streaming for responsiveness; fallback providers for uptime during outages.
Document processing pipelines	Batch inference overnight for large volumes at lower cost.
High-traffic support bots	Caching common questions; retries with backoff for rate-limit resilience.
Multi-team enterprise AI platforms	API gateways centralizing rate limits, routing, and logging across teams.

6. Practical Examples

Beginner: Spotting cacheable queries

For a support chatbot, list 3 query types likely to repeat often (good caching candidates) and 3 unlikely to repeat (poor caching candidates).

Beginner: Real-time vs. batch

Classify 5 hypothetical AI tasks as best suited to real-time, batch, or async inference, with reasoning for each.

Intermediate: Designing a backoff schedule

Design a retry schedule (max retries, wait times) for a batch job processing 10,000 items, balancing total runtime against rate-limit safety.

Advanced: Fallback chain design

Design a 3-tier fallback chain (primary → secondary → tertiary) for a production chatbot, specifying what triggers each fallback and any capability tradeoffs at each tier.

7. Code Examples

Python — Retry with exponential backoff

```python
# pip install anthropic
import anthropic
import time

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

def call_with_backoff(prompt, max_retries=4):
    for attempt in range(max_retries):
        try:
            return client.messages.create(
                model="claude-sonnet-4-6", max_tokens=200,
                messages=[{"role": "user", "content": prompt}]
            )
        except anthropic.RateLimitError:
            if attempt == max_retries - 1:
                raise  # out of retries, let it fail up to the caller
            wait = 2 ** attempt  # 1s, 2s, 4s, 8s...
            print(f"Rate limited, retrying in {wait}s...")
            time.sleep(wait)

response = call_with_backoff("Summarize this article in 2 sentences: ...")
print(response.content[0].text)
```

JavaScript — Simple cache + streaming pattern

```javascript
// npm install @anthropic-ai/sdk
import Anthropic from "@anthropic-ai/sdk";
const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

const cache = new Map();  // simple in-memory cache; use Redis in real production

async function cachedOrStream(prompt, onToken) {
  if (cache.has(prompt)) {
    onToken(cache.get(prompt));  // instant cached response
    return;
  }

  let fullResponse = "";
  const stream = await client.messages.stream({
    model: "claude-sonnet-4-6",
    max_tokens: 300,
    messages: [{ role: "user", content: prompt }],
  });

  stream.on("text", (chunk) => {
    fullResponse += chunk;
    onToken(chunk);  // stream to the user as it arrives
  });

  await stream.finalMessage();
  cache.set(prompt, fullResponse);  // cache for next time
}
```

> **Tip**
> Combine retries, caching, and fallbacks in the same request path where appropriate — e.g., try cache first, then primary model with backoff, then fallback model on repeated failure.

8. Common Mistakes

- **Retrying immediately with no backoff**, worsening rate-limit situations instead of resolving them.
- **Caching genuinely dynamic or personalized queries** where reuse produces incorrect results.
- **Skipping streaming for interactive, user-facing features**, hurting perceived responsiveness for no reason.
- **Defaulting to real-time inference for large, non-urgent batch workloads**, wasting money and adding complexity.
- **Never testing fallback paths**, discovering they're broken during an actual outage.
- **Only testing at small scale**, assuming linear scaling without validating under real load.

9. Best Practices

- **Implement retries with exponential backoff** and a sensible maximum retry count from the start.
- **Cache genuinely repeated queries**, with clear expiration and invalidation rules.
- **Stream all interactive, user-facing text generation.**
- **Match inference mode to actual urgency**: real-time only when truly needed, batch/async otherwise.
- **Build and regularly test fallback paths**, not just at initial setup.
- **Load-test before scaling traffic**, and validate reliability continuously via Module 17's monitoring.

10. Hands-on Activities

Easy 5 exercises

1. Explain exponential backoff in your own words with a simple numeric example.

2. List 3 good caching candidates and 3 poor ones for a project from Module 14.

3. Classify 5 tasks as real-time, batch, or async, with reasoning.

4. Explain why streaming doesn't reduce total generation time but still helps.

5. Sketch the retry-with-backoff diagram from Section 4 from memory.

Intermediate 3 exercises

1. Build the Python retry-with-backoff function from Section 7 and test it against a simulated failure.

2. Add a simple caching layer to a project from Module 14 and measure the latency/cost difference on repeated queries.

3. Add streaming to a chat-style project and compare the user experience before and after.

Challenge 2 exercises

1. Design and implement a fallback system that switches to a secondary provider (Module 12) after 2 consecutive failures on the primary.

2. Design a full production architecture (on paper) for a high-traffic AI feature combining caching, retries, streaming, and fallbacks — explain how each piece interacts.

11. Mini Project

Goal

Harden a project from Module 14 with production-grade reliability patterns: retries with backoff, caching, and a basic fallback.

Requirements

- An Anthropic API key (and optionally a second provider for fallback testing)
- Python or JavaScript
- An existing project from Module 14

Step-by-Step Instructions

1. Wrap your project's model calls with retry-and-backoff logic (Section 7).
2. Add a simple cache for any genuinely repeatable queries in your project.
3. If your project is chat-style, add streaming for the response.
4. Add a fallback: if the primary call fails after all retries, switch to a secondary model/provider (or simulate this with a mock).
5. Test all four patterns explicitly: simulate a rate-limit error, a repeated query, and a total primary failure.

Expected Output

```
Test 1 (rate limit simulated): retried 2x with backoff, succeeded ✓
Test 2 (repeated query): served from cache, 0.01s (vs 1.4s uncached) ✓
Test 3 (primary total failure): fell back to secondary provider ✓
Test 4 (chat query): streamed token-by-token ✓
```

Possible Improvements

- Move caching to a real persistent store (Redis) instead of in-memory.
- Add Module 17-style tracing to log every retry and fallback activation.
- Load-test the hardened version against the original to measure the reliability improvement.

12. Review Questions

Multiple Choice (10)

1. LLMOps is best described as:

A) A single tool B) The discipline of operationalizing LLM applications in production C) A prompting technique D) A model family

2. Exponential backoff means:

A) Retrying instantly every time B) Progressively increasing wait time between retry attempts C) Never retrying D) Reducing token count

3. Caching is most valuable when:

A) Every query is unique B) Queries genuinely repeat C) Never useful for LLMs D) Only for images

4. Streaming primarily improves:

A) Total generation time B) Perceived latency for interactive use C) Token cost to zero D) Model accuracy

5. Batch inference is best suited for:

A) Interactive real-time chat B) Large-volume, non-urgent workloads C) Single one-off questions D) Nothing useful

6. A model fallback activates when:

A) Everything is working normally B) The primary model/provider fails or is unavailable C) Never D) Only during testing

7. An API gateway's main role is to:

A) Replace the model entirely B) Centralize cross-cutting concerns like auth, rate limits, and routing C) Increase temperature D) Train new models

8. Reliability and scalability are best described as:

A) A single specific technique B) Emergent properties resulting from techniques like retries, caching, and fallbacks applied together C) Irrelevant to AI systems D) Only about server hardware

9. Why test fallback paths regularly, even when the primary is healthy?

A) It's unnecessary B) A broken, untested fallback provides no protection during a real outage C) It increases cost with no benefit D) It's required by law

10. Using real-time synchronous calls for a large, non-urgent overnight workload is:

A) Optimal B) Often wasteful compared to batch inference C) Required D) The only option available

Identification (5)

1. _____ is the discipline of operationalizing LLM applications for reliable production use.

2. _____ progressively increases wait time between retry attempts.

3. _____ stores and reuses previous results to avoid redundant model calls.

4. _____ returns a model's response incrementally, token by token, as it's generated.

5. _____ is a pre-configured secondary model/provider a system switches to on primary failure.

True or False (5)

1. Retrying instantly and repeatedly with no backoff is the recommended approach to rate limits.

2. Streaming reduces the total time it takes a model to generate a full response.

3. Batch inference is often cheaper than real-time inference for large, non-urgent workloads.

4. Fallback paths should be tested regularly, not just set up once and left untested.

5. API gateways are most valuable once multiple applications/teams share model access.

Situational (5)

1. Your batch job crashes entirely the first time it hits a rate limit. What pattern should you add?

2. Your support bot repeatedly answers the same top-10 questions, each costing a fresh API call. What should you add?

3. Users complain your chat interface "feels slow" even though total response time is reasonable. What UX change would help most?

4. You need to process 50,000 documents overnight with no urgency. What inference mode fits best?

5. Your primary AI provider had an outage last month and your app went completely down. What should you add to prevent this next time?

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-B, 5-B, 6-B, 7-B, 8-B, 9-B, 10-B

Identification: 1-LLMOps, 2-Exponential backoff, 3-Caching, 4-Streaming, 5-Model fallback

True or False: 1-False, 2-False, 3-True, 4-True, 5-True

Situational (sample reasoning):

1. Retries with exponential backoff.
2. Caching for the frequently repeated questions.
3. Streaming — showing text as it generates instead of waiting for the full response.
4. Batch inference.
5. A model fallback to a secondary provider.

13. Cheat Sheet

🚦 Rate Limits

Retries + exponential backoff. Cap max retries and total wait time.

💾🏎️ Cost & Perceived Speed

Caching — repeated queries.
Streaming — interactive UX.

🤔⏳ Inference Modes

Real-time (interactive) · Batch (large, non-urgent) · Async (submit now, check later).

🔀🏗️ Resilience

Fallbacks for provider outages. Test them regularly — reliability is techniques applied together.

14. Key Takeaways

- LLMOps is the umbrella discipline for running LLM applications reliably in production, building on Modules 16-17.
- Retries with exponential backoff turn transient failures into recoverable hiccups instead of hard crashes.
- Caching avoids redundant model calls for genuinely repeated queries — a high-leverage cost/latency win.
- Streaming improves perceived latency for interactive use, even without changing total generation time.
- Batch and async inference trade immediacy for cost and throughput — the right default for large, non-urgent workloads.
- Model fallbacks keep applications running through provider outages — but only if regularly tested.
- Reliability and scalability are emergent properties of these techniques applied together, validated continuously via monitoring (Module 17).

15. Additional Resources

Official Documentation

- Anthropic — rate limits, streaming, and batch API documentation (docs.claude.com)

Related Modules

- Module 09 — Prompt Optimization (cost/latency fundamentals this module operationalizes)
- Module 17 — LLM Observability & Monitoring (the visibility layer validating these patterns work)
- Module 12 — AI APIs (provider/platform choices relevant to fallback design)

Communities & Further Learning

- Traditional DevOps/SRE resources on retries, circuit breakers, and reliability engineering — most concepts transfer directly to LLMOps.